

Questão 08 - Postmortem técnico de incidente em produção

Framework escolhido: R-I-S-E

Justificativa da escolha (resumo): o cenário exige correlacionar múltiplas fontes de evidência heterogêneas, descartar hipóteses por eliminação e emitir uma recomendação binária (rollback vs. scaling) sob restrição de tempo. O R-I-S-E é o único dos cinco frameworks que separa explicitamente o processo analítico (Steps) do critério de qualidade do resultado (Expectation), o que é decisivo quando a IA precisa demonstrar o raciocínio de descarte antes de concluir. Comparação detalhada com T-A-G e C-A-R-E na seção de justificativa.

Prompt construído (R-I-S-E)

[ROLE]

Você é um SRE Sênior atuando como Incident Commander técnico da Hill Valley Tech, responsável por produzir, durante um incidente em andamento, um postmortem técnico preliminar para Doc Brown (CTO), que está em call agora e precisa decidir em até 20 minutos entre duas ações concretas: (A) rollback do deploy chronos-api v2.48.0, ou (B) scaling emergencial (aumento de limits do RDS e do pool de conexões). Seu papel não é apenas descrever o incidente, é fundamentar uma recomendação objetiva com base nas evidências fornecidas.

[INPUT]

1) Changelog do deploy mais recente (ontem, 18:42 UTC):

Deploy chronos-api: v2.47.0 -> v2.48.0

Argo CD sync: 2026-04-23 18:42:11 UTC

Changelog:

- Adicionado endpoint POST /v2/transactions/batch
- Refatorado cliente do Ledger (pool de conexões movido para nova biblioteca interna)
- Bump de psycopg 3.1.18 -> 3.2.0
- Reduzido timeout do Ledger de 5s para 2s

2) Métricas do Beacon (últimos 30min):

timestamp | p99_latency_ms | req_rate_s | err_rate_pct

13:30 | 420 | 1200 | 0.2

13:45 | 510 | 1450 | 0.3
14:00 | 780 | 1780 | 0.8
14:10 | 2400 | 2100 | 4.5
14:15 | 5200 | 2400 | 8.2
14:20 | 8100 | 2650 | 11.7

3) Log do pod chronos-api-79c4d8b9-xk2jp:

14:19:48 [ERROR] [ledger-client] connection pool exhausted (max=20, active=20, waiting=147)
14:19:49 [WARN] [ledger-client] query timeout after 2000ms: SELECT ... FROM transactions WHERE ...
14:19:49 [ERROR] [handler] POST /v2/transactions/batch failed: context deadline exceeded
14:19:50 [ERROR] [ledger-client] connection reset by peer
14:19:51 [WARN] [circuit-breaker] ledger-client OPEN (threshold 50%, current 87%)
14:19:52 [ERROR] [reactor] failed to publish message: chronos-api upstream error

4) Estado do Reactor (fila chronos-transactions):

- 50.127 mensagens acumuladas, crescendo a ~800/min
- Consumer lag atual: 18 minutos e aumentando

5) Estado do cluster:

- Chronos: 12/12 pods running (HPA no máximo)
- CPU médio dos pods: 62%
- Memória média dos pods: 71%
- Conexões ativas ao Ledger: 240/250 (limite do RDS)

[STEPS]

Execute a análise nesta ordem, tornando cada etapa visível no documento final:

1. Construa uma linha do tempo correlacionando o horário do deploy (18:42 UTC de ontem) com o início da degradação nas métricas — identifique se a degradação começou logo após o deploy ou é mais recente/relacionada a

pico de tráfego

2. Cruze o changelog do deploy com os erros do log: avalie especificamente se a redução do timeout do Ledger (5s→2s) e a refatoração do pool de conexões são causa provável do "connection pool exhausted (max=20)" e do circuit breaker aberto
3. Descarte ou confirme explicitamente saturação de CPU/memória/HPA como causa raiz, usando os números do estado do cluster (62% CPU, 71% memória, 12/12 pods) — isto é decisivo para saber se scaling horizontal resolveria algo
4. Trate o acúmulo na fila do Reactor (50.127 mensagens, lag de 18min) como efeito ou causa — justifique com base na ordem dos eventos no log (o erro do reactor aparece antes ou depois dos erros do ledger-client?)
5. Avalie as duas ações candidatas (rollback vs. scaling emergencial) individualmente contra a causa raiz identificada: cada uma resolve o problema? Em quanto tempo? Que risco cada uma carrega?
6. Emita uma recomendação objetiva (não "depende") com nível de confiança, e liste as ações imediatas de mitigação, na ordem em que devem ser executadas

[EXPECTATION]

O documento precisa ser lido e compreendido por Doc Brown em menos de 3 minutos dentro da call — seja direto, sem rodeios narrativos. A recomendação final precisa ser inequívoca (uma ação clara, não as duas opções em pé de igualdade) e sustentada explicitamente pelos números fornecidos, nunca por suposição não verificável. Se a evidência for insuficiente para descartar totalmente uma hipótese, isso deve ser declarado como incerteza explícita, não omitido.

[FORMATO DE SAÍDA]

Markdown, seções: Timeline | Causa Raiz | Ligação Causal (evidência) | Rollback vs. Scaling (tabela comparativa) | Recomendação | Ações Imediatas.

Execução 1 — Claude Sonnet 4.5 (Anthropic)

Modelo: claude-sonnet-4-5

Provedor: Anthropic

Justificativa de escolha: Modelo de referência do curso, com forte capacidade de raciocínio analítico encadeado e fidelidade a prompts estruturados com múltiplos blocos. Escolhido como baseline por ser o modelo utilizado nas demais questões do desafio, permitindo consistência na avaliação dos frameworks.

Output gerado

Postmortem Técnico Preliminar — Incidente Chronos API (14:20 UTC, 2026-04-24)

Timeline

- ****2026-04-23 18:42 UTC**** — Deploy v2.47.0 → v2.48.0 via Argo CD.
Mudanças críticas: refatoração do pool de conexões do Ledger e timeout reduzido de 5s para 2s.
- ****13:30–14:00 UTC (hoje)**** — Sistema estável, crescimento orgânico de tráfego (1200→1780 req/s), p99 em 420–780ms, erro <1%.
- ****14:10 UTC**** — p99 salta para 2400ms, erro sobe para 4.5%. Início claro da degradação, ~19h30min após o deploy.
- ****14:19:48–52 UTC**** — Cascata de falhas: pool de conexões esgotado (max=20, 147 em fila) → timeouts → circuit breaker abre (87%) → Reactor falha ao publicar mensagens.
- ****14:20 UTC**** — p99 em 8100ms, erro em 11.7%, fila do Reactor com 50.127 mensagens e lag de 18min.

****Observação-chave:**** a degradação não ocorreu no momento do deploy, mas ~19h depois, ao atingir carga de pico. O deploy introduziu uma vulnerabilidade latente que só se manifestou sob volume elevado.

Causa Raiz

****Pool de conexões subdimensionado (max=20) combinado com timeout excessivamente agressivo (2s), introduzidos pela refatoração do v2.48.0.****

Ligação Causal (evidência)

1. `connection pool exhausted (max=20, active=20, waiting=147)` — 14:19:48
2. `query timeout after 2000ms` — 14:19:49 (consequência: conexões presas)
3. `POST /v2/transactions/batch failed: context deadline exceeded` — 14:19:49
4. `circuit-breaker OPEN (87%)` — 14:19:51
5. `reactor failed to publish message` — 14:19:52 (**último da cadeia**)

CPU (62%) e memória (71%) não saturados. HPA já no teto (12/12).

O gargalo é exclusivamente o pool de conexões ao Ledger — não capacidade computacional.

O acúmulo do Reactor é **efeito**, não causa: o erro do reactor aparece após o circuit breaker abrir, provando que o Reactor falhou por consequência do Chronos estar rejeitando processamento.

Rollback vs. Scaling Emergencial

Critério	Rollback (v2.47.0)	Scaling emergencial
----------	--------------------	---------------------

Ataca a causa raiz?	Sim	Não
Tempo de mitigação	5–10 min (Argo CD rollback já testado)	Incerto
Risco	Baixo	Médio-alto
Perda de funcionalidade	Temporária: endpoint `/v2/transactions/batch` indisponível	Nenhuma
Reversibilidade	Alta	Baixa sob incidente ativo

Recomendação

Rollback imediato para v2.47.0. Confiança: alta.

CPU e memória não são o gargalo — scaling não resolve. A causa raiz está diretamente mapeada em duas mudanças do v2.48.0 (timeout 2s + novo pool), e o rollback é a ação de menor risco e maior reversibilidade dentro dos 20 minutos disponíveis.

Ações Imediatas

1. `argocd app rollback chronos-api <revisão v2.47.0>`
2. Monitorar pool de conexões e p99 nos 5 minutos seguintes
3. ****Não escalar RDS**** — evidência não sustenta essa hipótese
4. Reactor deve drenar a fila naturalmente após estabilização; monitorar consumer lag
5. Abrir investigação assíncrona sobre a nova biblioteca de pool antes de reintroduzir o v2.48.0

Execução 2 — GPT-4o (OpenAI)

Modelo: gpt-4o (versão 2024-11-20)

Provedor: OpenAI

Justificativa de escolha: GPT-4o é o modelo flagship atual da OpenAI, com capacidade de raciocínio analítico comparável ao Claude Sonnet em tarefas de correlação de múltiplas evidências. Escolhido como segundo provedor para atender ao requisito do desafio de utilizar ao menos dois provedores distintos entre OpenAI, Anthropic, Google e Meta/Ollama, e por ser amplamente adotado em contextos de SRE e DevOps, tornando a comparação de outputs relevante para prática profissional.

Output

Postmortem Técnico Preliminar — Chronos API | 2026-04-24 14:20 UTC

Timeline

- ****2026-04-23 18:42 UTC**** — Deploy v2.48.0 via Argo CD. Alterações de risco: refatoração do cliente Ledger (nova lib de pool) e timeout reduzido de 5s → 2s.
- ****14:00 UTC**** — Tráfego acelera (1780 req/s). Primeira elevação de p99 (780ms) e err_rate (0.8%) — sinais iniciais sob carga crescente.
- ****14:10 UTC**** — Degradação se instala: p99 = 2400ms, err_rate = 4.5%. Conexões ao Ledger começam a se esgotar.
- ****14:19:48 UTC**** — Pool de conexões esgotado (max=20, waiting=147). Timeout de 2s provoca retenção de conexões, acelerando o colapso do pool.
- ****14:19:51 UTC**** — Circuit breaker abre com 87% de falha.
- ****14:19:52 UTC**** — Reactor falha ao publicar: efeito em cascata confirmado.
- ****14:20 UTC**** — Estado crítico: p99=8100ms, 11.7% de erro, 50k mensagens acumuladas no Reactor.

Causa Raiz

****Subdimensionamento do pool de conexões ao Ledger (max=20) combinado com timeout reduzido a 2s, introduzidos pela refatoração do v2.48.0, gerando esgotamento de conexões sob carga de pico.****

Conexões que antes toleravam até 5s de latência no Ledger agora expiram em 2s, mas permanecem presas no pool até liberação — acelerando o esgotamento.

Ligação Causal (evidência)

| Horário | Evento | Significado |

| --- | --- | --- |

| 14:19:48 | Pool exhausted: max=20, waiting=147 | Causa direta — pool insuficiente |

| 14:19:49 | Query timeout 2000ms | Timeout agressivo retendo conexões |

| 14:19:49 | batch endpoint: context deadline exceeded | Impacto no endpoint novo |

| 14:19:51 | Circuit breaker OPEN (87%) | Sistema de proteção ativado |

| 14:19:52 | Reactor falha | Efeito em cascata — último da cadeia |

****CPU 62% e memória 71%: sem saturação. HPA no teto (12/12): scaling horizontal já esgotado e ineficaz para este gargalo. Causa é de conexão, não de compute.****

Rollback vs. Scaling Emergencial

| | Rollback v2.47.0 | Scaling RDS + pool |

| --- | --- | --- |

| Resolve causa raiz? | ****Sim**** — reverte timeout e pool problemático | Não — aumenta limite sem corrigir comportamento da lib |

| Tempo até mitigação | ~5–8 min | 15–30 min (parameter group RDS pode exigir reboot) |

| Risco | Baixo — versão comprovada | Médio — sem garantia de estabilização |

| Funcionalidade perdida | `/v2/transactions/batch` temporariamente indisponível | Nenhuma |

| Reversibilidade | Total | Parcial durante incidente |

Recomendação

****Executar rollback imediato para v2.47.0. Confiança: alta.****

Scaling não endereça a causa raiz: o pool de 20 conexões e o timeout de 2s são decisões de código, não de infraestrutura. Aumentar o limite do RDS não

altera o comportamento da nova biblioteca de pool. O rollback é mais rápido, reversível e diretamente mapeado à causa identificada.

Ações Imediatas

1. ``argocd app rollback chronos-api <revision-id do v2.47.0>``
2. Confirmar queda do `err_rate` e do `p99` em até 10 minutos
3. Não alterar limites do RDS neste momento
4. Monitorar consumer lag do Reactor — deve cair gradualmente após o Chronos se estabilizar
5. Pós-incidente: investigar a nova biblioteca de pool de conexões
(max configurável? comportamento sob timeout? leak de conexões?)
6. Reintroduzir v2.48.0 apenas após validação em ambiente de carga controlada

Comparação entre outputs

Aspecto	Claude Sonnet 4.5	GPT-4o
Aderência ao formato solicitado	Total — seções exatas como pedidas	Total — leve variação nos títulos, manteve estrutura
Diagnóstico de causa raiz	Idêntico — pool + timeout do v2.48.0	Idêntico — mesmo diagnóstico com ênfase no mecanismo de retenção de conexão
Descarte de CPU/memória	Explícito no corpo do texto	Explícito, com destaque para "causa é de conexão, não de compute"
Causa vs. efeito do Reactor	Justificado pela ordem dos eventos no log	Confirmado via tabela causal com horários
Recomendação final	Rollback, confiança alta	Rollback, confiança alta
Diferença notável	Mais narrativo na timeline; destaca a vulnerabilidade latente introduzida pelo deploy	Mais tabular no raciocínio causal; acrescenta pergunta investigativa no pós-incidente (max configurável? leak?)

Conclusão da comparação: ambos os modelos chegaram à mesma recomendação com nível de confiança equivalente. O GPT-4o apresentou o raciocínio causal de forma mais tabular e adicionou perguntas de investigação pós-incidente mais detalhadas; o Claude Sonnet 4.5 foi mais preciso na narrativa da linha do tempo e no destaque da vulnerabilidade latente como conceito explicativo da janela de ~19h entre deploy e falha.

Justificativa estendida — R-I-S-E e comparação com outros frameworks

Como R-I-S-E aparece no prompt

Componente	Onde aparece	Por quê
Role	"SRE Sênior atuando como Incident Commander... seu papel não é apenas descrever o incidente, é fundamentar uma recomendação"	Ancora a IA numa postura de decisão , não de relatório passivo — essencial porque Doc Brown precisa de uma posição, não de uma lista neutra de fatos
Input	Os 5 artefatos completos (changelog, métricas, log, fila, cluster) organizados e rotulados	Sem estruturar o Input explicitamente, a IA tende a tratar as fontes de forma solta; aqui elas viram blocos de evidência endereçáveis, o que facilita a etapa seguinte de correlação
Steps	6 passos ordenados: timeline → cruzamento causal → descarte de hipótese (CPU/memória) → causa vs. efeito da fila → avaliação das duas ações → recomendação	É o componente mais crítico deste prompt — obriga a IA a descartar explicitamente a hipótese de saturação computacional (passo 3) antes de recomendar, evitando o erro mais comum em diagnóstico sob pressão: escalar recursos sem confirmar que é isso que está faltando
Expectation	"lido em menos de 3 minutos... recomendação inequívoca... sustentada pelos números... incerteza declarada, não omitida"	Traduz a restrição real do cenário (20 minutos, decisão ao vivo) em critério de qualidade do texto — impede que a IA entregue um "depende" elegante, que seria inútil numa call de incidente

Comparação com T-A-G

O que se ganharia: T-A-G é mais enxuto — Action cobriria os mesmos passos analíticos com menos estrutura de prompt. Para um incidente com causa raiz óbvia, seria suficiente.

O que se perderia: T-A-G não tem componente Role nativo. Aqui, a persona de "Incident Commander que precisa recomendar, não só descrever" muda o tom do output — sem ela, o modelo tende a produzir um relatório mais descritivo e menos assertivo. Além disso, o Goal do T-A-G expressa uma finalidade única, enquanto aqui existem duas ações mutuamente exclusivas para comparar. Essa avaliação comparativa explícita (rollback vs. scaling) se encaixa melhor no Expectation do R-I-S-E, que pode exigir "recomendação inequívoca, não as duas opções em pé de igualdade" — no T-A-G essa exigência ficaria diluída no Goal, com risco de a IA apresentar as duas opções sem se comprometer.

Comparação com C-A-R-E

O que se ganharia: o Context do C-A-R-E absorveria bem os 5 artefatos, e o Result capturaria o critério de sucesso ("postmortem que Doc Brown consegue ler em 3 minutos e decide com base nele").

O que se perderia: o componente Example não tem para onde apontar — não existe um postmortem anterior fornecido como padrão de referência. Forçar um Example genérico (ex.: "escreva como um postmortem do Google SRE Book") adicionaria ruído sem ganho real. Mais importante: sem Steps explícitos — C-A-R-E tem só Action, um bloco único — corre-se o risco de a IA pular direto para a recomendação sem demonstrar o raciocínio de descarte de hipóteses, que é justamente o que dá credibilidade técnica ao postmortem quando Doc Brown pode questionar "por que não é só falta de recurso?". C-A-R-E é o framework certo quando há consistência de padrão a preservar (como no módulo Terraform da Q06), não quando o desafio é diagnóstico analítico original.

Por que R-T-F e B-A-B foram descartados rapidamente

- **R-T-F** é bom demais para artefatos autocontidos e simples (Dockerfile, script) — não tem componente para lidar com **múltiplas fontes de evidência correlacionadas** nem para impor uma sequência de descarte de hipóteses. Usá-lo aqui produziria um prompt raso demais para a complexidade analítica exigida.
- **B-A-B** pressupõe uma transição de estado com destino único e já conhecido (como modernizar o Deployment do Chronos, Questão 5). Aqui, o "After" não está definido de antemão — é justamente o que a análise precisa **descobrir** (rollback ou scaling?). Forçar B-A-B tenderia a enviesar a IA para prescrever uma correção específica antes mesmo de comparar as duas alternativas, o que compromete a neutralidade analítica que Doc Brown precisa para decidir.